

Metanet 测试用例设计

本文档为 Metanet 设计文档体系的第四层：测试用例设计。

文档体系：- [整体设计](#) — 两产品生态、三层架构、界面划分 - [概念设计](#) — 产品定位、核心理念、设计原则 - [系统设计](#) — 节点架构、合约、支付通道 - [详细设计](#) — 共识、挖矿、结算协议细节 - 测试设计 (本文档) — 测试用例设计

设计章节交叉引用：系统设计章节（如“第五节”）见 [2-SystemDesign](#)，详细设计章节（如“第一节”）见 [3-DetailedDesign](#)。

设计原则

- 1. 三段式格式: 每个测试用例采用”前置条件 → 操作 → 期望结果”
- 2. 标签分类: `[unit]` 单元测试 | `[edge]` 边界条件 | `[integration]` 集成测试 | `[property]` 属性不变量 | `[security]` 安全性
- 3. 覆盖率目标: 每个 package ≥80% 行覆盖率
- 4. 命名映射: 规范 ID `T{N}. {M}. {K}` → 代码函数名 `Test{Category}_{Scenario}`
- 5. 独立性: 每个测试可独立运行，无测试间依赖
- 6. 确定性: 所有测试使用固定种子/mock，禁止随机行为
- 7. 表驱动: Go 惯例，使用 `[]struct{ name string; ... } + t.Run()`
- 8. 断言框架: `testify/require` (致命错误) 与 `testify/assert` (非致命错误) 区分使用

总览

#	类别	设计参照	代码文件	目标
T1	存储合约	系统设计五, 详细设计一	<code>metanet_chain/ storage_deal_test.go</code>	2
T2	存储证明	系统设计五, 详细设计二	<code>metanet_chain/ storage_proof_test.go</code>	3

#	类别	设计参照	代码文件	目标
T3	ECDH 双层加密	系统设计五, 详细设计三	metanet_chain/ ecdh_test.go	1
T4	支付与检索	系统设计七, 详细设计四-五	metanet_chain/ payment_test.go	4
T5	合并挖矿与热数据	系统设计二, 详细设计七	metanet_chain/ mining_test.go	2
	合计			12

T1. 存储合约

设计参照: 系统设计第五节, 详细设计第一节 代码文件: metanet_chain/storage_deal_test.go

ID	用例名称	前置条件	操作	期望结果	标签
T1.1	存储合约创建: N 个 UTXO 对应 N 期	Owner 与 Metanet Node 协商完成	创建 StorageDeal 交易	合约交易包含 N 个输出, 各含 expected_proof_hash (预计算), 锁定 MNT Token	[unit]
T1.2	确定性挑战计算	已知 contract_txid	challenge_k = SHA256(contract_txid k) (k=1..N)	挑战值确定性可复现, 不同 k 产生不同 challenge, 合约创建时即可预计算所有期	[property]

T2. 存储证明

设计参照: 系统设计第五节, 详细设计第二节 代码文件: metanet_chain/storage_proof_test.go

ID	用例名称	前置条件	操作	期望结果	标签
T2.1		Metanet Node 持有正确数据	构建 StorageProof	Script 验证通过 (chunk_data +	[unit]

ID	用例名称	前置条件	操作	期望结果	标签
	Metanet Node 提交正确 Merkle proof		交易 → Script 执行	merkle_siblings → merkle_root 匹配), Metanet Node 领取该 期奖励	
T2.2	Metanet Node 提交错误 proof	Metanet Node 伪造数据	构建错误 StorageProof → Script 执行	Script 验证失败, merkle_root 不匹配, UTXO 不可花费	[security]
T2.3	Metanet Node 提交其他 chunk 的 proof	Metanet Node 持有数据但提交 错误 chunk	chunk_index 与挑战不匹配	验证失败, chunk_index 对应的 expected_proof_hash 不匹配	[security]

T3. ECDH 双层加密

设计参照: 系统设计第五节, 详细设计第三节 代码文件: `metanet_chain/ecdh_test.go`

ID	用例名称	前置条件	操作	期望结果	标签
T3.1	ECDH 双层加密	Owner 有加密文 件, Metanet Node 已注册	Owner ECDH 重加 密 → Metanet Node 接收	Provider 密文 != Owner 密文, Metanet Node 无 法解密原始内容 (仅持有外层密钥), Owner 可通过两层 密钥解密	[unit]

T4. 支付与检索

设计参照: 系统设计第七节, 详细设计第四-五节 代码文件: `metanet_chain/payment_test.go`

ID	用例名称	前置条件	操作	期望结果	标签
T4.1	x402 检索支付		User 支付 BSV →	Metanet Node 正 确返回解密后的数	[integration]

ID	用例名称	前置条件	操作	期望结果	标签
		Metanet Node 持有数据, User 请求下载	Metanet Node 返回数据	据, BSV 支付交易有效	
T4.2	Token 支付通道: 开启	Owner 与 Metanet Node 协商	创建 2-of-2 多签 funding 交易	funding 交易正确创建, 双方各持一份签名	[unit]
T4.3	Token 支付通道: 更新	通道已开启	双方签署新状态	sequence_number 递增, 新余额分配正确, 旧状态作废	[unit]
T4.4	Token 支付通道: 正常关闭	通道有多次状态更新	双方协商关闭	最终余额按最新状态正确分配, 无需等待时间锁	[unit]

T5. 合并挖矿与热数据

设计参照: 系统设计第二节, 详细设计第七节 代码文件: `metanet_chain/mining_test.go`

ID	用例名称	前置条件	操作	期望结果	标签
T5.1	合并挖矿	BTC/BSV 矿工参与	矿工在 coinbase 嵌入 Metanet Chain block hash	Metanet Chain 验证 AuxPoW, 区块有效, 安全性随算力增长	[integration]
T5.2	热数据 CDN: Metanet Node 缓存热门内容后服务 x402 请求	Metanet Node 自愿缓存热门文件	User 通过 x402 请求数据	Metanet Node 正确返回数据并收取 BSV 费用, 不需要存储合约	[integration]

交叉引用索引

测试类别	系统设计章节	详细设计章节	代码路径
T1 存储合约	五 (冷数据: Archive 合约模式)	一 (存储合约 Bitcoin Script)	metanet_chain/ storage_deal_test.go
T2 存储证明	五 (冷数据: Archive 合约模式)	二 (存储证明 Bitcoin Script)	metanet_chain/ storage_proof_test.go
T3 ECDH 双层加密	五 (冷数据: Archive 合约模式)	三 (ECDH 双层加密流程)	metanet_chain/ ecdh_test.go
T4 支付与检索	七 (x402 支付通道)	四-五 (支付通道协议, HTTP 扩展)	metanet_chain/ payment_test.go
T5 合并挖矿与热数据	二 (Metanet Chain 基本设计)	七 (合并挖矿技术细节)	metanet_chain/ mining_test.go